Prob. 1 – Rounding errors

Ex 3 – K.P. Lind

It is known that, for smooth enough functions, the central finite difference approximation of second-order derivatives converges quadratically, that is,

$$\left|\frac{u(x-h)-2u(x)+u(x+h)}{h^2}-u''(x)\right| = O(h^2)\,.$$

Computationally, however, this is only attainable for a certain range of h. If we keep reducing h, eventually the convergence will break down, since *machine accuracy* (or *machine precision* depending on the source) is usually limited to $O(10^{-16})$ values. Based on that, estimate the mesh size h (in order of magnitude) for which roundoff and approximation errors become comparable.

$$\left|\frac{u(x-h)-2u(x)+u(x+h)}{h^2}-u''(x)\right| = O(h^2)$$

$$|Ch^2| \approx \left|\varepsilon/h^2\right| \Rightarrow h \approx \sqrt[4]{\left|\varepsilon/C\right|} = O(\sqrt[4]{\varepsilon}) = O(\sqrt[4]{10^{-16}}) = \underline{\underline{10^{-4}}}$$

Prob. 2 – Rounding errors

In this problem we are going to see how round off errors affect the solution to systems of linear equations.

Consider the problem of interpolating the polynomial $f(y) = 1 + y^2$ at the points $y = 0, 1, 2$, using a second degree polynomial. This can be done by solving the system of equations,

$$Ax = b, \quad A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix},$$

a) (J) Solve the system $Ax = b$ on your computer and compare the solution to $(1, 0, 1)$, using the relative error,

$$\frac{|A^{-1}b - (1,0,1)|}{|(1,0,1)|},$$

where $|\cdot|$ is the Euclidean norm.

```
A = np.array([
    [1, 0, 0],
    [1, 1, 1],
    [1, 2, 3]])

Ainv = np.linalg.inv(A)

b = np.array([1.0, 2.0, 5.0])

x = np.linalg.solve(A, b)

Ainvb = Ainv * b

B = np.array([1, 0, 1])

lengthUp = np.linalg.norm(Ainvb - B, 2)     # Øvre brøkstrek
lengthDown = np.linalg.norm(B, 2)           # Nedre brøkstrek


relativeError = lengthUp / lenghtDown

print(f'The solution to the system Ax = b is x = {x}')
print(f'The relative errror is {relativeError}')
```

```
The solution to the system Ax = b is x = [ 1. -1.  2.]
The relative errror is 7.445257102083687
```

b) Compute the condition number of the matrix A (feel free to use any software to do so). Given that the relative error, $|e|/|b|$, in the right hand side is of order 10^{-10}, estimate the order of the relative error in the solution x.

$$\kappa(A) = \max_{b,e\neq 0} \frac{|A^{-1}e|}{|A^{-1}b|} / \frac{|e|}{|b|},$$

```
E = np.array([1e-10])

lenghtAinvE = np.linalg.norm(Ainv * E, 2)
lenghtAinvB = np.linalg.norm(Ainv * b, 2)
lenghtE = np.linalg.norm(E, 2)
lenghtB = np.linalg.norm(b, 2)

conditionNumber = (lenghtAinvE / lenghtAinvB) / (lenghtE / lenghtB)
print(f'The condition number of the matrix A is K(A) = {conditionNumber}.')
```

```
The condition number of the matrix A is K(A) = 2.387486005612572.
```

c) (J) Change the interpolation points to $0, 0.01, 2$, and make a new system of linear equations for the interpolating polynomial's coefficients. Solve this system on your computer and compare this with $(1, 0, 1)$. Finally compute the condition number of this system's matrix. Still using a relative error in b of order 10^{-10}, what is an upper bound for the order of the relative error in the solution now?

d) Experiment numerically with different interpolation points in the interval $[0, 2]$, and see if you are able to get an error, $|(1, 0, 1) - A^{-1}b|$ of order 10^{-5} or bigger.

Prob. 3 – Fixed-point method

Ex 3 – K.P. Lind

In this exercise, we will use fixed-point iterations to find the real root of

$$f(x) = 2x^3 - x^2 + 2x - 1 = 0\,. \tag{1}$$

To apply the fixed-point algorithm, we first need to rewrite Eq. (1) as $x = g(x)$, but this representation is not unique. For example, we can write

$$x = \frac{-2x^3 + x^2 + 1}{2} := g_1(x)\,, \quad \text{or} \quad x = \sqrt[3]{\frac{x^2 - 2x + 1}{2}} := g_2(x)\,.$$

In fact, the algorithm's convergence will depend on whether we choose g_1 or g_2 as our $g(x)$. We will experiment with both versions, and you can use the Jupyter notebook 05-*Nonlinear-eqs.ipynb* for help.

a) Find the expressions for $g_1'(x)$ and $g_2'(x)$ and, based on that, answer: would you rather select g_1 or g_2 as your $g(x)$? Why?

QUOTIENT RULE

$$g_1'(x) \overset{\text{QUOTIENT RULE}}{=} \frac{(-2x^3 + x^2 + 1)' \cdot 2 - (-2x^3 + x^2 + 1) \cdot 2'}{2^2} = x(1 - 3x)$$

CHAIN & QUOTIENT RULE

$$g_2'(x) \overset{\text{CHAIN \& QUOTIENT RULE}}{=} \left(\sqrt[3]{u(x)}\right)' \cdot u'(x)\,, \quad u(x) = \frac{x^2 - 2x + 1}{2} = \frac{1}{3u^{2/3}} \cdot (x - 1) = \frac{x - 1}{3\left(\frac{x^2 - 2x + 1}{2}\right)^{2/3}}$$

I would choose g_1 as $g(x)$, because $g_1'(x)$ is a much simpler expression than $g_2'(x)$.

b) (J) Set $x^{(0)} = 0.9$ and $g(x) = g_1(x)$, and iterate until $|x^{(k+1)} - x^{(k)}| < 10^{-6}$. What is the solution obtained, and how many iterations are needed to reach that?

Calculated with FN04-Numerical solution of nonlinear eq.

$x^{(0)} = 0.9$ and $g(x) = g_1(x)$

```
k =  0, x =   0.9000000000
k =  1, x =   0.1760000000
k =  2, x =   0.5100362240
k =  3, x =   0.4973892073
k =  4, x =   0.5006458997
k =  5, x =   0.4998381076
k =  6, x =   0.5000404469
k =  7, x =   0.4999898866
k =  8, x =   0.5000025282
k =  9, x =   0.4999993679
The last step was below the tolerance( 3.1603e-06<1.0000e-05)

x = 0.49999936793423405 after 9 iterations
```

c) (J) Now, do the same for $g(x) = g_2(x)$. How many iterations are now needed?

Calculated with FN04-Numerical solution of nonlinear eq.

$x^{(0)} = 0.9$ and $g(x) = g_2(x)$

```
k =  0, x =   0.9000000000
k =  1, x =   0.0707106781
k =  2, x =   0.6571067812
k =  3, x =   0.2424621202
k =  4, x =   0.5356601718
k =  5, x =   0.3283378413
k =  6, x =   0.4749368671
k =  7, x =   0.3712757018
k =  8, x =   0.4445752147
k =  9, x =   0.3927446321
k = 10, x =   0.4293943885
k = 11, x =   0.4034790972
k = 12, x =   0.4218039755
k = 13, x =   0.4088463298
k = 14, x =   0.4180087689
k = 15, x =   0.4115299461
k = 16, x =   0.4161111656
k = 17, x =   0.4128717542
k = 18, x =   0.4151623640
k = 19, x =   0.4135426583
k = 20, x =   0.4146879632
k = 21, x =   0.4138781103
k = 22, x =   0.4144507628
k = 23, x =   0.4140458364
k = 24, x =   0.4143321626
k = 25, x =   0.4141296994
k = 26, x =   0.4142728625
k = 27, x =   0.4141716309
k = 28, x =   0.4142432124
k = 29, x =   0.4141925966
k = 30, x =   0.4142283874
k = 31, x =   0.4142030795
k = 32, x =   0.4142209749
k = 33, x =   0.4142083209
k = 34, x =   0.4142172686
The last step was below the tolerance( 8.9477e-06<1.0000e-05)

x = 0.41421726862948566 after 34 iterations
```

Prob. 4 – Fixed-point iterations

Ex 3 – K.P. Lind

In pipeline design for oil transport, pressure losses must be carefully estimated. They are directly proportional to a positive friction factor f_D, whose inverse square root $x := 1/\sqrt{f_D}$ is given by a non-linear equation. In fact, it is known that x depends only on the Reynolds number R (look up *"Reynolds number"* online, if you are interested in fluid mechanics). For a turbulent flow at a given R, the equation to find x is

$$x + 1.93 \ln(1.32x/R) = 0\,,$$

in which $\ln(x)$ denotes the natural logarithm. In this exercise, we will set $R = 5000$ and use fixed-point iterations to approximate x, according to

$$x^{(k+1)} = g(x^{(k)}), \quad \text{with } g(x) := -1.93 \ln(1.32x/5000)\,. \tag{2}$$

Let us also define the iteration error as $e_k := |x^{(k)} - x^{(k-1)}|$. Based on that, you are asked to:

a) Compute $g'(x)$ and use the result to determine whether $g(x)$, $x > 0$, is an increasing, decreasing or a non-monotonic function.

CHAIN RULE

$$g'(x) = (-1.93 \ln(u(x)))' \cdot u'(x)\,, \quad u(x) = \frac{1.32x}{5000} \Rightarrow g(x) = -\frac{1.93}{u(x)} \cdot \frac{1.32}{5000} = -\frac{1.93}{x}$$

We can see that $g'(x) < 0 \quad \forall x > 0$, which means that $f(x)$ is a decreasing function.

b) Calculate the maximum and minimum values of $g(x)$ in the interval $x \in [e, e^3]$.

From a) we take notice that $g'(x)$ gets smaller as x gets bigger. Then we know that $g(x)$ has its biggest and smallest value respectively when $x = e$ and $x = e^3$.

$$g_{max} = g(e) = -1.93 \ln(1.32e/5000) = \underline{\underline{13.97}}$$

$$g_{min} = g(e^3) = -1.93 \ln(1.32e^3/5000) = \underline{\underline{10.11}}$$

c) Determine the maximum value L of $|g'(x)|$ in the interval $[e, e^3]$.

We know from earlier that $g'(x)$ has its biggest value when $x = e$

$$L = |g'(x)| = g'(e) = -\frac{1.93}{e} = -0.7100073215$$

From lectures, we know that the smaller L is, the faster FPM converges.

d) For $x^{(0)} = e^2$, perform the first fixed-point iteration for the solution of Eq. (2).

$$x^{(k+1)} = g(x^{(k)})$$

$$x^{(1)} = g(x^{(0)}) = g(e^2) = -1.93 \ln(1.32e^2/5000) = \underline{\underline{12.04}}$$

e) Based on the values of L, $x^{(0)}$ and $x^{(1)}$, find an upper bound for the number of iterations required to reach a tolerance of 10^{-3}.

```
k =  0, x =  12.0423536078
k =  1, x =  11.0996838947
k =  2, x =  11.2570045120
k =  3, x =  11.2298418305
k =  4, x =  11.2345044675
The last step was below the tolerance( 4.6626e-03<1.0000e-02)

x = 11.23450446748468 after 4 iterations
```

$\Rightarrow$ OK after 4 iterations when tol $= 10^{-3}$.